The Problem: Legacy Systems

What are Legacy System? Legacy systems are old softwares, technologies, tools, or infrastructures used by an organization. They were often introduced ages ago and might be based on outdated technologies, but they continue to provide essential functions, and that's why they've stuck around.

Challenges with Legacy Systems:

1. **Technical Debt:** Over the years, patches, fixes, and new features might have been added haphazardly, leading to a complex, hard-to-maintain codebase.
2. **Obsolete Technology:** The technology stack may no longer be in vogue, making it difficult to find experts or support.
3. **Integration Issues:** Modern tools and services might not easily integrate with older systems, causing silos of information.
4. **Scalability Concerns:** Legacy systems might not be designed to handle today's scale or might not be cloud-ready.
5. **Costly Maintenance:** As the system ages, maintenance can become more expensive, both in terms of time and resources.
6. **Security Vulnerabilities:** Older systems may not have been designed with current security best practices in mind, making them susceptible to attacks.

Why is Replacing Legacy Systems Hard?

1. **Critical Operations:** Many legacy systems are deeply embedded in an organization's critical operations. Any disruption can have significant impacts.
2. **Migration Risks:** Data migration from old to new systems can be fraught with risks like data loss or corruption.
3. **Training & Adoption:** Staff might be used to the old system. Introducing a new system requires training and can face resistance.
4. **High Costs:** Building a new system and ensuring it works as reliably as the old one can be expensive.
5. **Unknown Unknowns:** There may be undocumented features or behaviors in the legacy system that users have come to rely upon.

Strangler Fig Pattern to the Rescue:

Here is how the strangler fig design pattern helps us replacing the legacy system:

1. **Incremental Replacement:** Instead of a big bang approach, replace the legacy system piece by piece. This reduces risks and allows for iterative feedback.
2. **Coexistence:** The new and old systems run side-by-side. As features are moved to the new system, traffic is rerouted accordingly, ensuring operations continue without disruption.
3. **Feedback Loops:** By gradually transitioning users, feedback can be obtained early and often. This ensures the new system meets user needs.
4. **Reduced Risk:** If a part of the new system has issues, it's easier to revert changes or fix them without affecting the entire system.
5. **Flexibility:** The pattern provides flexibility in adopting new technologies. As the industry evolves, the new system can evolve with it.
6. **End Goal:** Over time, the legacy system's functionality is entirely replaced by the new system. Once the old system has no more active users or features, it can be safely retired.